

Assignment 5
ISAN 5355 with Dr. Zhao
John Courtright

Original Table:

A table with the students and their grades on different topics.								
UnitID	StudentID	Date	TutorID	Topic	Room	Grade	Book	TutEmail
U1	St1, St4	23.02.03	Tut1	GMT	629	4.7, 4.3	Deumlich	tut1@fhbb.ch
U2	St1	18.11.02	Tut3	Gln	631	5.1	Zehnder	tut3@fhbb.ch
U5	St2	05.05.03	Tut3	PhF	632	4.9	Dümmmlers	tut3@fhbb.ch
U4	St2	04.07.03	Tut5	AVQ	621	5	SwissTopo	tut5@fhbb.ch

Variables:

- ❖ UnitID : Unique identifier for each instructional course session
- ❖ StudentID : Unique ID for each student participating in a session
- ❖ Date : Date of session
- ❖ TutorID : Unique ID for the tutor hosting the unit
- ❖ Topic : Subject or topic covered in session
- ❖ Room : Room # where session was held
- ❖ Grade : Student's grade received for the session
- ❖ Book : Book or learning material associated with session
- ❖ TutEmail: Email address of the tutor hosting the session

Create Database

```
-- Create database
CREATE DATABASE assign5;

-- Set collation and default character set for db
ALTER DATABASE assign5 CHARSET utf8mb4
                        COLLATE utf8mb4_0900_ai_ci;

-- Set storage engine to be InnoDB
-- Will apply to newly created tables for the new db
SET SESSION default_storage_engine = InnoDB;

-- Activate schema
USE assign5;
```

We're going to take this original table and take it through **normalization** steps to produce 1st, 2nd, and 3rd normalization forms of the database structure.

Original Table Creation Script

```
CREATE TABLE 0NF
(
    UnitID VARCHAR(10),
    -- Two students in 0NF U1
    StudentID VARCHAR(10),
    -- DD-MM-YY format
    Date VARCHAR(10),
    TutorID VARCHAR(10),
    Topic VARCHAR(10),
    -- Room could be INT, but we shouldn't need to do any computation with it
    Room VARCHAR(10),
    -- In 0NF, U1 has two grades, separated by comma
    Grade VARCHAR (10),
    Book VARCHAR(20),
    TutEmail VARCHAR(30)
);

-- Insert original records
INSERT INTO 0NF (UnitID, StudentID, Date, TutorID, Topic, Room, Grade, Book,
TutEmail)
VALUES ('U1', 'St1, St4', '23.02.03', 'Tut1', 'GMT', '629', '4.7, 4.3',
'Deumlich', 'tut1@fhbb.ch');
INSERT INTO 0NF (UnitID, StudentID, Date, TutorID, Topic, Room, Grade, Book,
TutEmail)
VALUES ('U2', 'St1', '18.11.02', 'Tut3', 'Gin', '631', '5.1', 'Zehnder',
'tut3@fhbb.ch');
INSERT INTO 0NF (UnitID, StudentID, Date, TutorID, Topic, Room, Grade, Book,
TutEmail)
VALUES ('U3', 'St2', '05.05.03', 'Tut3', 'PhF', '632', '4.9', 'Dummlerw',
'tut3@fhbb.ch');
INSERT INTO 0NF (UnitID, StudentID, Date, TutorID, Topic, Room, Grade, Book,
TutEmail)
VALUES ('U1', 'St2', '04.07.03', 'Tut5', 'AVQ', '621', '5', 'SwissTopo',
'tut5@fhbb.ch');
```

0NF (Original) Table in MySQL

```
51 -- Validate
52 • SELECT *
53 FROM 0NF;
```

100% 92:49

Result Grid Filter Rows: Search Export:

UnitID	StudentID	Date	TutorID	Topic	Room	Grade	Book	TutEmail
U1	St1, St4	23.02.03	Tut1	GMT	629	4.7, 4.3	Deumlich	tut1@fhbb.ch
U2	St1	18.11.02	Tut3	Gin	631	5.1	Zehnder	tut3@fhbb.ch
U3	St2	05.05.03	Tut3	PhF	632	4.9	Dummlerw	tut3@fhbb.ch
U1	St2	04.07.03	Tut5	AVQ	621	5	SwissTopo	tut5@fhbb.ch

0NF 1 Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
6	13:05:17	INSERT INTO 0NF (UnitID, StudentID, Date, TutorID, Topic, Room, Grade, Book, TutEmail) VA...	Error Code: 1145: Table 'assignment' doesn't exist	0.000 sec
7	13:05:18	CREATE TABLE 0NF (UnitID VARCHAR(10), -- Two students in 0NF U1 StudentID VAR...	0 row(s) affected	0.063 sec
8	13:05:21	INSERT INTO 0NF (UnitID, StudentID, Date, TutorID, Topic, Room, Grade, Book, TutEmail) VA...	1 row(s) affected	0.011 sec
9	13:05:21	INSERT INTO 0NF (UnitID, StudentID, Date, TutorID, Topic, Room, Grade, Book, TutEmail) VA...	1 row(s) affected	0.0015 sec
10	13:05:21	INSERT INTO 0NF (UnitID, StudentID, Date, TutorID, Topic, Room, Grade, Book, TutEmail) VA...	1 row(s) affected	0.00089 sec
11	13:05:21	INSERT INTO 0NF (UnitID, StudentID, Date, TutorID, Topic, Room, Grade, Book, TutEmail) VA...	1 row(s) affected	0.00047 sec
12	13:05:49	SELECT * FROM 0NF	4 row(s) returned	0.00040 sec / 0.000...

In 0NF form, this table has a few issues. *StudentID* can contain multiple students (U1 -> St1, St4). *Grade* contains multiple grades (U1 -> 4.7, 4.3). If the session (each row) has multiple students, it stores their ID and grade in a **comma separated string** rather than an **atomic value**.

To convert this table to 1NF, we'll need to **break the multivalued fields into multiple rows**. Each row in the 1NF table will represent a session and the individual student was in that session. The *StudentID* and *Grade* columns will only hold one value that matches their respective student. We're creating some more problems with UnitID and StudentID both being a primary key and partial dependencies (some rows only pertaining to Unit and not Student, or vice versa), but we'll solve those later.

1NF Table Creation Script

```
CREATE TABLE 1NF (  
    UnitID      VARCHAR(10),  
    StudentID   VARCHAR(10),  
    Date        VARCHAR(10),  
    TutorID     VARCHAR(10),  
    Topic       VARCHAR(10),  
    Room        VARCHAR(10),  
    -- Changing grade to be numeric  
    Grade       DECIMAL(2,1),  
    Book        VARCHAR(20),  
    TutEmail    VARCHAR(30)  
);  
  
INSERT INTO 1NF VALUES  
(  
'U1', 'St1', '23.02.03', 'Tut1', 'GMT', '629', 4.7, 'Deumlich',  
'tut1@fhbb.ch'),  
(  
'U1', 'St4', '23.02.03', 'Tut1', 'GMT', '629', 4.3, 'Deumlich',  
'tut1@fhbb.ch'),  
(  
'U2', 'St1', '18.11.02', 'Tut3', 'Gin', '631', 5.1, 'Zehnder',  
'tut3@fhbb.ch'),  
(  
'U3', 'St2', '05.05.03', 'Tut3', 'PhF', '632', 4.9, 'Dummlerw',  
'tut3@fhbb.ch'),  
(  
'U4', 'St2', '04.07.03', 'Tut5', 'AVQ', '621', 5.0, 'SwissTopo',  
'tut5@fhbb.ch');
```

81 -- Validate
82 • SELECT *
83 FROM 1NF;
84
85

100% 1:80

Result Grid Filter Rows: Search Export:

UnitID	StudentID	Date	TutorID	Topic	Room	Grade	Book	TutEmail
U1	St1	23.02.03	Tut1	GMT	629	4.7	Deumlich	tut1@fhbb.ch
U1	St4	23.02.03	Tut1	GMT	629	4.3	Deumlich	tut1@fhbb.ch
U2	St1	18.11.02	Tut3	Gin	631	5.1	Zehnder	tut3@fhbb.ch
U3	St2	05.05.03	Tut3	PhF	632	4.9	Dummlerw	tut3@fhbb.ch
U4	St2	04.07.03	Tut5	AVQ	621	5.0	SwissTopo	tut5@fhbb.ch

1NF 3 Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
21	13:34:51	CREATE TABLE 1NF (UnitID VARCHAR(10), StudentID VARCHAR(10), Date V...	0 row(s) affected	0.011 sec
22	13:34:54	INSERT INTO 1NF VALUES ('U1', 'St1', '23.02.03', 'Tut1', 'GMT', '629', 4.7, 'Deumlich', 'tut1@f...	5 row(s) affected Records: 5 Duplicates: 0 Warnings...	0.0061 sec
23	13:35:23	SELECT * FROM 1NF	5 row(s) returned	0.00066 sec / 0.000...

We've now converted the original table to 1NF. *StudentID* and *Grade* now just represent a single student. Each row represents a student in a tutoring session.

However, we now have an issue where we have a composite key with *UnitID* and *StudentID*. We can create two separate tables: one where each record is a tutoring session with the relevant variables and one where each record represents a student using the UnitID as a foreign key. Separating this information will also solve our issue of partial dependencies in our records.

2NF Tables Creation Script

```
CREATE TABLE Unit2NF (  
    UnitID      VARCHAR(10) PRIMARY KEY,  
    Date        VARCHAR(10),  
    TutorID     VARCHAR(10),  
    Topic        VARCHAR(10),  
    Room        VARCHAR(10),  
    Book        VARCHAR(20)  
);  
  
-- Student Table  
CREATE TABLE StudentUnit2NF (  
    UnitID      VARCHAR(10),  
    StudentID   VARCHAR(10),  
    Grade       DECIMAL(2,1),  
    - Declare both as primary keys to avoid student-unit duplication  
    PRIMARY KEY (UnitID, StudentID),  
    FOREIGN KEY (UnitID) REFERENCES Unit2NF(UnitID)  
);  
  
-- Insert Records into both tables  
INSERT INTO Unit2NF VALUES  
( 'U1', '23.02.03', 'Tut1', 'GMT', '629', 'Deumlich'),  
( 'U2', '18.11.02', 'Tut3', 'Gin', '631', 'Zehnder'),  
( 'U3', '05.05.03', 'Tut3', 'PhF', '632', 'Dummlerw'),  
( 'U4', '04.07.03', 'Tut5', 'AVQ', '621', 'SwissTopo');  
  
INSERT INTO StudentUnit2NF VALUES  
( 'U1', 'St1', 4.7),  
( 'U1', 'St4', 4.3),  
( 'U2', 'St1', 5.1),  
( 'U3', 'St2', 4.9),  
( 'U4', 'St2', 5.0);
```

Unit2NF, StudentUnit2NF Tables and Response

	UnitID	Date	TutorID	Topic	Room	Book
	U1	23.02.03	Tut1	GMT	629	Deumlich
	U2	18.11.02	Tut3	Gin	631	Zehnder
	U3	05.05.03	Tut3	PhF	632	Dummlerw
	U4	04.07.03	Tut5	AVQ	621	SwissTopo
	NULL	NULL	NULL	NULL	NULL	NULL

	UnitID	StudentID	Grade
	U1	St1	4.7
	U1	St4	4.3
	U2	St1	5.1
	U3	St2	4.9
	U4	St2	5.0
	NULL	NULL	NULL

✓ 24	14:17:54	CREATE TABLE Unit2NF (UnitID VARCHAR(10) PRIMARY KEY, Date VARCHAR(10)...	0 row(s) affected	0.071 sec
✓ 25	14:17:54	CREATE TABLE StudentUnit2NF (UnitID VARCHAR(10), StudentID VARCHAR(10),...	0 row(s) affected	0.017 sec
✓ 26	14:17:54	INSERT INTO Unit2NF VALUES ('U1', '23.02.03', 'Tut1', 'GMT', '629', 'Deumlich'), ('U2', '18.11...	4 row(s) affected Records: 4 Duplicates: 0 Warnings...	0.0082 sec
✓ 27	14:17:54	INSERT INTO StudentUnit2NF VALUES ('U1', 'St1', 4.7), ('U1', 'St4', 4.3), ('U2', 'St1', 5.1), ('...	5 row(s) affected Records: 5 Duplicates: 0 Warnings...	0.0022 sec
✓ 28	14:20:53	SELECT * FROM Unit2NF	4 row(s) returned	0.027 sec / 0.00039...
✓ 29	14:20:53	SELECT* FROM StudentUnit2NF	5 row(s) returned	0.00026 sec / 0.0000...

We now have a 2NF table structure for our tutoring session records. Table *Unit2NF* represents each unit, where each row is a different unit. We include the date, the TutorID for the tutor that hosted the session, the topic, room, and book used. These columns do not have anything to do with the student, so we separate them out. *StudentUnit2NF* represents the students that received tutoring. Each row is a different student, using *StudentID* as the primary key. We also store what unit that student attended (the same UnitID can be used for multiple students) and their grade for the session.

To normalize into 3rd normal form, we need to ensure no transitive dependency, or that our non-key columns don't have a dependency on another column.

TutEmail is in the *Unit2NF* table and uses UnitID as its primary key. However, a tutor's email would depend on the tutor, in this case *TutorID*, so we'll need to create another *Tutor* table.

3NF Table Creation Script

```
CREATE TABLE Tutor3NF (  
    TutorID    VARCHAR(10) PRIMARY KEY,  
    TutEmail   VARCHAR(30)  
);  
  
CREATE TABLE Unit3NF (  
    UnitID     VARCHAR(10) PRIMARY KEY,  
    Date       VARCHAR(10),  
    TutorID    VARCHAR(10),  
    Topic      VARCHAR(10),  
    Room       VARCHAR(10),  
    Book       VARCHAR(20),  
    FOREIGN KEY (TutorID) REFERENCES Tutor3NF(TutorID)  
);  
  
CREATE TABLE StudentUnit3NF (  
    UnitID     VARCHAR(10),  
    StudentID  VARCHAR(10),  
    Grade      DECIMAL(2,1),  
    PRIMARY KEY (UnitID, StudentID),  
    FOREIGN KEY (UnitID) REFERENCES Unit3NF(UnitID)  
);  
  
-- Insert Records  
INSERT INTO Tutor3NF VALUES  
('Tut1', 'tut1@fhbb.ch'),  
('Tut3', 'tut3@fhbb.ch'),  
('Tut5', 'tut5@fhbb.ch');  
  
INSERT INTO Unit3NF VALUES  
('U1', '23.02.03', 'Tut1', 'GMT', '629', 'Deumlich'),  
('U2', '18.11.02', 'Tut3', 'Gin', '631', 'Zehnder'),  
('U3', '05.05.03', 'Tut3', 'PhF', '632', 'Dummlerw'),  
('U4', '04.07.03', 'Tut5', 'AVQ', '621', 'SwissTopo');  
  
INSERT INTO StudentUnit3NF VALUES  
('U1', 'St1', 4.7),  
('U1', 'St4', 4.3),  
('U2', 'St1', 5.1),  
('U3', 'St2', 4.9),  
('U4', 'St2', 5.0);
```

3NF Tables and Response

UnitID	Date	TutorID	Topic	Room	Book
U1	23.02.03	Tut1	GMT	629	Deumlich
U2	18.11.02	Tut3	Gin	631	Zehnder
U3	05.05.03	Tut3	PhF	632	Dummlerw
U4	04.07.03	Tut5	AVQ	621	SwissTopo
NULL	NULL	NULL	NULL	NULL	NULL
Unit3NF 6		StudentUnit3NF 7		Tutor3NF 8	

UnitID	StudentID	Grade
U1	St1	4.7
U1	St4	4.3
U2	St1	5.1
U3	St2	4.9
U4	St2	5.0
NULL	NULL	NULL
Unit3NF 6		StudentUnit3NF 7

TutorID	TutEmail
Tut1	tut1@fhbb.ch
Tut3	tut3@fhbb.ch
Tut5	tut5@fhbb.ch
NULL	NULL
Unit3NF 6	
StudentUnit3NF 7	
Tutor3NF 8	

✓ 41	14:39:04	CREATE TABLE Tutor3NF (TutorID VARCHAR(10) PRIMARY KEY, TutEmail VARCHAR(10)...	0 row(s) affected
✓ 42	14:39:04	CREATE TABLE Unit3NF (UnitID VARCHAR(10) PRIMARY KEY, Date VARCHAR(10)...	0 row(s) affected
✓ 43	14:39:04	CREATE TABLE StudentUnit3NF (UnitID VARCHAR(10), StudentID VARCHAR(10),...	0 row(s) affected
✓ 44	14:39:04	INSERT INTO Tutor3NF VALUES ('Tut1', 'tut1@fhbb.ch'), ('Tut3', 'tut3@fhbb.ch'), ('Tut5', 'tut...	3 row(s) affected Records: 3 Duplicates: 0 Warnings...
✓ 45	14:39:04	INSERT INTO Unit3NF VALUES ('U1', '23.02.03', 'Tut1', 'GMT', '629', 'Deumlich'), ('U2', '18.1...	4 row(s) affected Records: 4 Duplicates: 0 Warnings...
✓ 46	14:39:04	INSERT INTO StudentUnit3NF VALUES ('U1', 'St1', 4.7), ('U1', 'St4', 4.3), ('U2', 'St1', 5.1), ('...	5 row(s) affected Records: 5 Duplicates: 0 Warnings...
✓ 47	14:40:19	SELECT * FROM Unit3NF	4 row(s) returned
✓ 48	14:40:19	SELECT * FROM StudentUnit3NF	5 row(s) returned
✓ 49	14:40:19	SELECT * FROM Tutor3NF	3 row(s) returned

With those steps, we were able to create a 3rd normalization form table set based on our original records. Each table has it's own primary key and represents its own "unit" (tutoring unit, student, tutor) where the features are solely dependent on the primary key and no other record. Values are atomic as well.

Let's test out the system by inserting 3 new records into each table (3 new sessions, 3 new students, 3 new tutors).

New Record Insert Queries

```
INSERT INTO Tutor3NF (TutorID, TutEmail) VALUES
('Tut7', 'tut7@fhbb.ch'),
('Tut8', 'tut8@fhbb.ch'),
('Tut9', 'tut9@fhbb.ch');
```

```
INSERT INTO Unit3NF (UnitID, Date, TutorID, Topic, Room, Book) VALUES
('U5', '12.03.04', 'Tut7', 'MLB', '640', 'Data Mining 101'),
('U6', '28.09.04', 'Tut8', 'STAT', '645', 'Applied Statistics'),
('U7', '15.01.05', 'Tut9', 'DSA', '650', 'Data Structures');
```

```
INSERT INTO StudentUnit3NF (UnitID, StudentID, Grade) VALUES
('U5', 'St10', 4.8),
('U6', 'St11', 5.2),
('U7', 'St12', 4.4);
```

SELECT Statement for all 3 Tables

```
SELECT
    su.UnitID,
    su.StudentID,
    u.Date,
    u.TutorID,
    u.Topic,
    u.Room,
    su.Grade,
    u.Book,
    t.TutEmail
FROM StudentUnit3NF su
JOIN Unit3NF u
    ON su.UnitID = u.UnitID
JOIN Tutor3NF t
    ON u.TutorID = t.TutorID
ORDER BY su.UnitID, su.StudentID;
```

Joined Tables

UnitID	StudentID	Date	TutorID	Topic	Room	Grade	Book	TutEmail
U1	St1	23.02.03	Tut1	GMT	629	4.7	Deumlich	tut1@fhbb.ch
U1	St4	23.02.03	Tut1	GMT	629	4.3	Deumlich	tut1@fhbb.ch
U2	St1	18.11.02	Tut3	Gin	631	5.1	Zehnder	tut3@fhbb.ch
U3	St2	05.05.03	Tut3	PhF	632	4.9	Dummlerw	tut3@fhbb.ch
U4	St2	04.07.03	Tut5	AVQ	621	5.0	SwissTopo	tut5@fhbb.ch
U5	St10	12.03.04	Tut7	MLB	640	4.8	Data Mining 101	tut7@fhbb.ch
U6	St11	28.09.04	Tut8	STAT	645	5.2	Applied Statistics	tut8@fhbb.ch
U7	St12	15.01.05	Tut9	DSA	650	4.4	Data Structures	tut9@fhbb.ch

Result 1

Action Output

	Time	Action	Response
1	15:40:09	SELECT su.UnitID, su.StudentID, u.Date, u.TutorID, u.Topic, u.Room, su.Gra...	8 row(s) returned

ERR Diagram of Database (inc 0NF, 1NF, 2NF, and 3NF)



